A1.6 DOCUMENTO DE ARQUITECTURA Y APROBACIÓN

PROGRAMA EDUCATIVO

Ingeniería en Sistemas Computacionales

NOMBRE DEL PROYECTO

Automatización de mantenimiento preventivo con monitoreo permanente utilizando APIs y scripts

EMPRESA

Infraestructura Digital Predictiva Bitlab, S.A.P.I. de C.V.

PRESENTA

Axel Gabriel Ramírez Ortiz
Matrícula: 122043202 · Grupo: S206

ASESOR UPQ:

Ana Laura Lira Cortes

CO-ASESOR UPQ:

Ivan Peredo Valderrama

ASESORA EMPRESARIAL:

M.T.A. Sandra Lucía de la Fuente Bermúdez

Querétaro, Qro., 29 de octubre del 2026

ÍNDICE

1. OBJETIVO DE LA ACTIVIDAD

2. DESARROLLO DE LA ACTIVIDAD

- Propuesta de cambio
- Contexto
- Requerimientos del módulo
- Topología de la solución
- Flujo general de atención de una falla
- Decisiones del diseño
- Riesgos del diseño

3. RESULTADOS DE LA ACTIVIDAD

4. APROBACIÓN

REFERENCIAS

1. OBJETIVO DE LA ACTIVIDAD

Con esta actividad junté en un solo documento todo lo que se definió en las actividades A1.1 a A1.5. La idea es que cualquiera que lo lea, empezando por la asesora externa, pueda ver cómo se conectan las partes de la solución, qué tiene que hacer el módulo, por qué se eligió cada tecnología y qué alternativas se dejaron fuera, que es justo lo que pide el criterio de aceptación del entregable E1.

La aprobación de este documento por parte de la asesora es el criterio de cierre de la fase F1 del protocolo. Con ella se da paso al desarrollo del agente.

Tabla 1. Datos de la actividad.

Concepto	Descripción
Clave de la actividad	A1.6
Fase del protocolo	F1. Requerimientos y arquitectura
Semana	S4 (28 de septiembre al 2 de octubre de 2026)
Horas planeadas	12 horas
Producto	E1. Requerimientos y arquitectura
Criterio de aceptación	Requerimientos por sistema operativo, catálogo de verificaciones, modelo de datos y contrato de la API, con decisiones registradas y alternativas descartadas.

Fuente: Protocolo de estadía profesional, Bitlab-IT (2026).

2. DESARROLLO DE LA ACTIVIDAD

Propuesta de cambio

Como el módulo se suma a un sistema que ya está en operación, lo primero fue dejar por escrito qué cambia, por qué y qué se espera obtener. Para eso usé una plantilla de propuesta de cambio, que se muestra en la Tabla 2.

Tabla 2. Propuesta de cambio del módulo de comandos remotos.

Campo	Descripción
Nombre del proyecto	BTL · Bitlab Terminal: módulo de comandos remotos para la plataforma de monitoreo de Bitlab-IT
Organización	Infraestructura Digital Predictiva Bitlab, S.A.P.I. de C.V.
Responsables	Axel Gabriel Ramírez Ortiz (practicante, desarrollo) · M.T.A. Sandra Lucía de la Fuente Bermúdez (asesora externa y líder del proyecto)

Campo	Descripción
Versión	1.0
Cambio propuesto	Agregar a la plataforma de monitoreo un módulo nuevo que permita: ejecutar a distancia comandos de un catálogo revisado en computadoras Windows y Mac; ver la respuesta del equipo en vivo en una consola; recibir un reporte del equipo cada 5 minutos y generar alertas de desempeño; mostrar, al seleccionar una alerta, los comandos que ayudan a resolverla y la sugerencia de un modelo de inteligencia artificial; y guardar el historial de todo lo ejecutado. El módulo usa sus propias tablas y no modifica lo que ya existe.
Por qué se requiere	Hoy la plataforma observa, pero no actúa. Cuando una computadora de una clínica falla, por ejemplo cuando deja de imprimir, un técnico tiene que conectarse o trasladarse para revisarla y correr los comandos a mano, y lo que hizo no queda registrado de forma uniforme. Con más clientes y el mismo personal, ese modelo no escala.
Resultado previsto	Que los técnicos atiendan la mayoría de las fallas de desempeño sin ir al sitio, con comandos revisados, viendo la respuesta en vivo y con un historial verificable de cada acción. Las metas son las del protocolo: 95 % de ejecuciones exitosas, 99 % de reportes entregados, menos de 60 segundos de latencia y menos del 5 % de procesador en el equipo.
Plazos estimados	F1 Requerimientos y arquitectura: 7 – 30 sep. F2 Agente de endpoint: 1 – 25 oct. F3 Ingesta, base de datos y panel: 26 oct – 18 nov. F4 Pruebas y optimización: 19 nov – 7 dic. F5 Reporte y cierre: 8 – 18 dic.
Factores adicionales	Tecnologías fijadas por el protocolo: Supabase (PostgreSQL), Lovable, PowerShell con Programador de tareas, zsh con launchd y Claude. El agente se instala solo con autorización del cliente y primero en laboratorio. Los accesos a la plataforma de Bitlab-IT se recibieron hasta la semana del 14 de octubre, por lo que el desarrollo arrancó en un proyecto propio de Supabase con la misma estructura.

Fuente: Elaboración propia.

Contexto

Como se explicó en el reporte de estadía, la plataforma de Bitlab-IT ya sabe qué equipos están en línea y cuáles tienen problemas, pero no puede hacer nada sobre ellos. Al analizar el problema, me quedó claro que la solución no era rehacer la plataforma, sino ponerle "manos": un agente en cada computadora que pueda ejecutar lo que el técnico decida, y un panel desde donde el técnico lo decida viendo la respuesta al momento. La Tabla 3 muestra cómo se reparte ese análisis en las actividades del E1.

Tabla 3. Actividades que integra este documento.

Actividad	Qué se definió	Pregunta que responde
A1.1 Requerimientos para Windows	PowerShell 5.1, política de ejecución, cuenta SYSTEM, Programador de tareas y requerimientos RF-W y RNF-W.	Pregunta 1
A1.2 Requerimientos para macOS	zsh, launchd, privacidad, firma y requerimientos RF-M y RNF-M.	Pregunta 1
A1.3 Catálogo de verificaciones	11 verificaciones, 11 tipos de alerta y 42 comandos iniciales.	Preguntas 1 y 4
A1.4 Modelo de datos	Diagrama entidad-relación, modelo relacional, índices y reglas de seguridad.	Pregunta 3
A1.5 Contrato de ingesta	Formato del reporte, funciones de la API, diagramas de secuencia y riesgos.	Pregunta 2

Fuente: Elaboración propia.

Requerimientos del módulo

Las actividades A1.1 y A1.2 definen lo que tiene que hacer el agente en cada sistema. Aquí faltaba la otra mitad: lo que tiene que hacer el módulo completo desde el punto de vista del técnico que lo usa. Las Tablas 4 y 5 reúnen esos requerimientos.

Tabla 4. Requerimientos funcionales del módulo.

Clave	Requerimiento	Descripción	Prioridad
RF-01	Iniciar sesión	El técnico entra al panel con su cuenta y solo ve los equipos de sus clientes.	Alta
RF-02	Ver equipos y su estado	Listar los equipos con su sistema, si están en línea y la hora de su último contacto.	Alta
RF-03	Ver alertas	Mostrar las alertas abiertas de cada equipo con su tipo y severidad.	Alta
RF-04	Menú de comandos por alerta	Al seleccionar una alerta, mostrar los comandos del catálogo que la atienden.	Alta
RF-05	Ejecutar un comando	Lanzar un comando del catálogo en un equipo seleccionado.	Alta
RF-06	Simular un comando	Ejecutar en modo de simulación para ver lo que haría sin aplicar cambios.	Alta
RF-07	Confirmar acciones que modifican el equipo	Pedir doble confirmación antes de ejecutar un comando destructivo.	Alta
RF-08	Ver la salida en vivo	Mostrar en una consola la respuesta del equipo mientras el comando se ejecuta.	Alta

Clave	Requerimiento	Descripción	Prioridad
RF-09	Consultar el historial	Ver las ejecuciones con fecha, equipo, técnico, comando, estado y salida, con filtros.	Alta
RF-10	Recibir sugerencias de IA	Mostrar qué comando del catálogo sugiere Claude para la alerta y por qué, sin ejecutarlo.	Media
RF-11	Administrar el catálogo	Dar de alta, editar y desactivar comandos del catálogo.	Media
RF-12	Programar comandos	Dejar un comando programado para que corra solo en un horario.	Media
RF-13	Ejecutar en varios equipos	Lanzar un mismo comando a varios equipos y ver el avance de cada uno.	Media
RF-14	Guardar comandos frecuentes	Que cada técnico tenga a la mano los comandos que más usa.	Baja
RF-15	Cancelar o reintentar	Detener una ejecución en curso o volver a lanzar una que falló.	Media

Fuente: Elaboración propia.

Tabla 5. Requerimientos no funcionales del módulo.

Clave	Tipo	Requerimiento	Meta o criterio
RNF-01	Confiabilidad	Las ejecuciones programadas deben terminar sin error.	95 % o más
RNF-02	Confiabilidad	Los reportes deben quedar guardados aunque la red falle por momentos (H2).	99 % o más
RNF-03	Rendimiento	El tiempo entre el fin de una ejecución y su aparición en el panel.	60 segundos o menos (p95)
RNF-04	Rendimiento	El agente casi no debe notarse en el equipo (H5).	Menos de 5 % de procesador y menos de 2 minutos
RNF-05	Usabilidad	Un técnico que no participó en el desarrollo debe poder instalar el agente con la guía.	10 minutos o menos por equipo
RNF-06	Seguridad	Ningún técnico debe ver ni ejecutar nada en equipos de otro cliente (H3).	Cero accesos cruzados
RNF-07	Seguridad	Solo se pueden ejecutar comandos del catálogo y toda acción destructiva tiene simulación.	Revisión de código y pruebas
RNF-08	Trazabilidad	Toda ejecución queda registrada con quién, cuándo, en qué equipo y con qué resultado.	100 % de las ejecuciones
RNF-09	Privacidad	El agente solo recolecta datos técnicos; nunca documentos, correos ni navegación.	Revisión del catálogo

Clave	Tipo	Requerimiento	Meta o criterio
RNF-10	Mantenibilidad	El módulo no modifica las tablas existentes y todos los cambios van en migraciones versionadas.	Revisión del repositorio
RNF-11	Compatibilidad	Usar solo las tecnologías del protocolo y las herramientas nativas de cada sistema.	Revisión de dependencias

Fuente: Elaboración propia.

Topología de la solución

Una vez claros los requerimientos, el siguiente paso fue decidir dónde vive cada parte y cómo se comunican. La Figura 1 muestra la topología. La organicé en capas: arriba, los equipos de los clientes con el agente; en medio, Supabase con su capa de servicios y su capa de datos; y abajo, la capa de presentación con el panel. Algo que cuidé mucho es que los equipos solo salen a internet por HTTPS; nadie se conecta a ellos desde afuera.

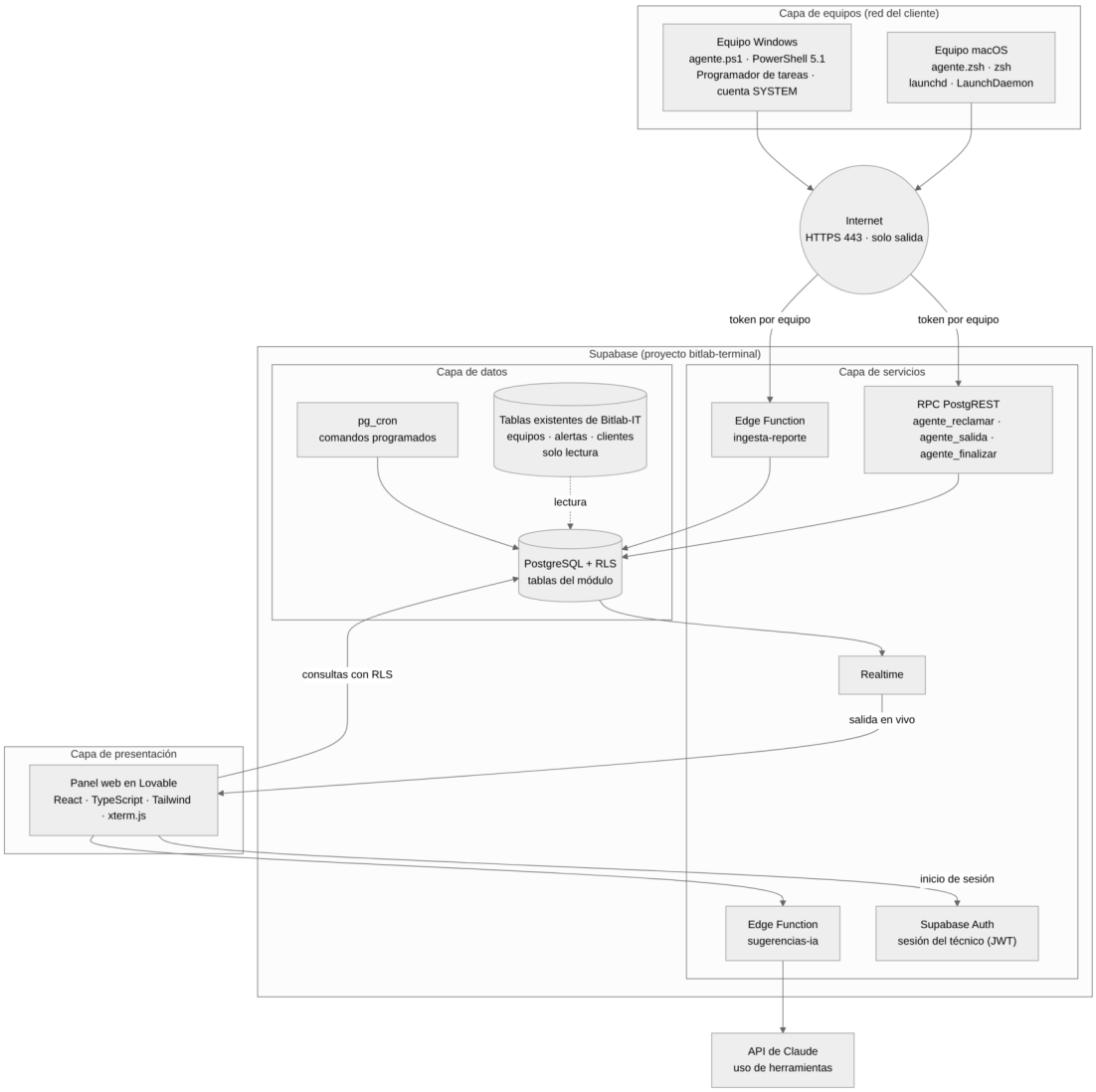


Figura 1. Topología de la solución por capas. *Fuente: Elaboración propia.*

Todas las piezas usan las tecnologías del protocolo, como se ve en la Tabla 6.

Tabla 6. Partes de la solución y su tecnología.

Parte	Tecnología	Qué hace
Agente para Windows	PowerShell 5.1 y Programador de tareas	Revisa el equipo, ejecuta los comandos del catálogo y envía los resultados.
Agente para Mac	zsh y launchd	Hace lo mismo y envía la información con el mismo formato.
Función de ingesta	Edge Function de Supabase	Recibe y revisa los reportes, y abre o cierra alertas.
Funciones del agente	Funciones de PostgreSQL	Entregan los comandos pendientes y reciben la salida.

Parte	Tecnología	Qué hace
Base de datos	Supabase sobre PostgreSQL, con reglas de seguridad	Guarda el catálogo, las ejecuciones, la salida, los reportes y la auditoría por cliente.
Tiempo real	Supabase Realtime	Manda al panel cada línea de salida en cuanto llega.
Comandos programados	pg_cron de Supabase	Lanza comandos de forma automática según un horario.
Panel	Lovable (React, TypeScript y Tailwind) con xterm.js	Menú de comandos por alerta, consola en vivo, catálogo e historial.
Sugerencias de IA	Edge Function con la API de Claude	Sugiere comandos del catálogo para cada alerta, sin ejecutarlos.
Organización del trabajo	GitHub (código y wiki) y Jira (proyecto BTL)	Control de versiones, revisión de código, documentación y seguimiento.

Fuente: Elaboración propia.

Flujo general de atención de una falla

Para cerrar el análisis quise ver el módulo desde los ojos del técnico, de principio a fin. La Figura 2 muestra ese flujo: desde que una computadora de la clínica empieza a fallar hasta que la alerta se cierra y todo queda en el historial. Los diagramas de secuencia con el detalle técnico de la ejecución de un comando y del envío de reportes están en la actividad A1.5.

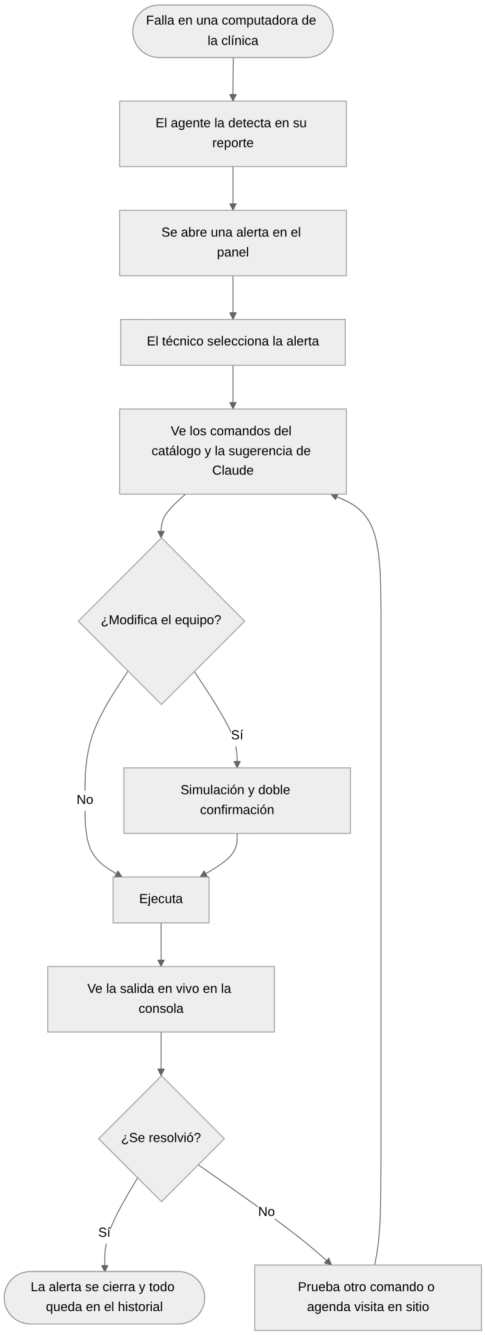


Figura 2. Diagrama de flujo general de atención de una falla. *Fuente: Elaboración propia.*

El diseño de las pantallas que sigue este flujo ya está en el prototipo «Bitlab Terminal», que se va a pasar a Lovable durante la fase F3.

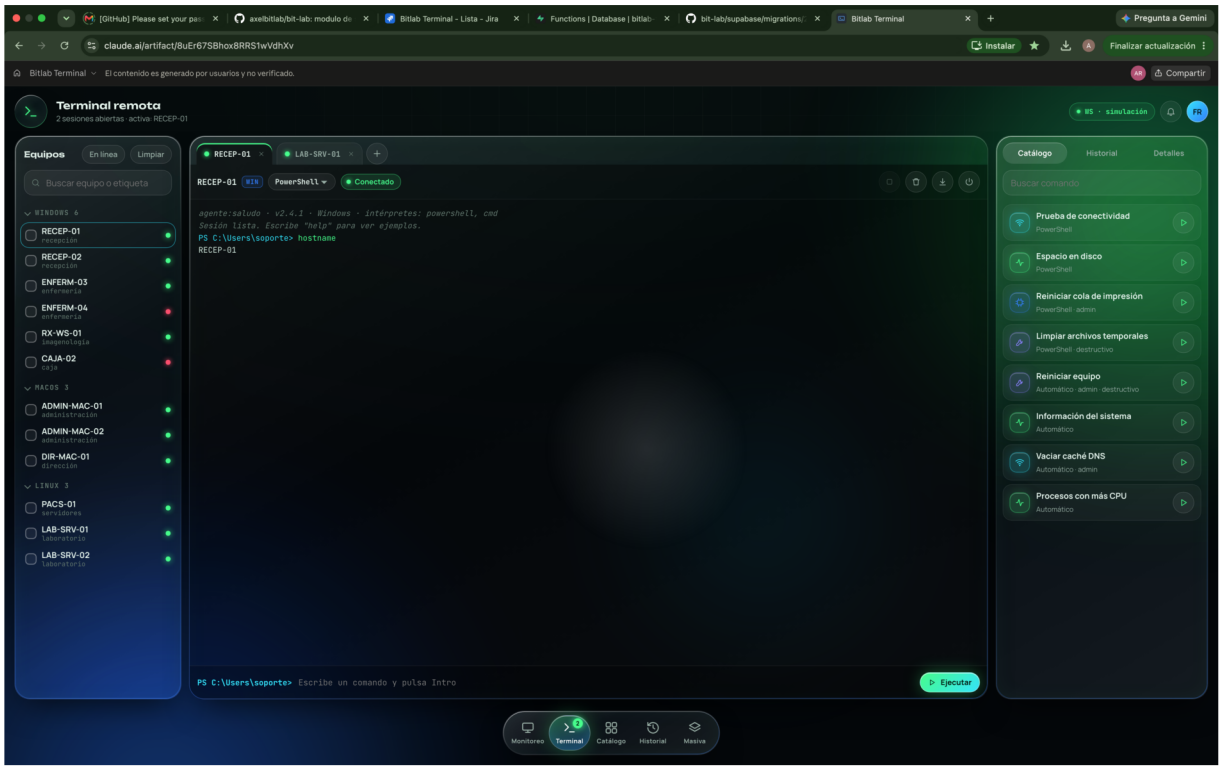


Figura 3. Prototipo de interfaz del módulo «Bitlab Terminal». *Fuente: Elaboración propia.*

Decisiones del diseño

A lo largo de la fase tomé varias decisiones que marcan el rumbo del proyecto. Para cada una dejé por escrito qué se decidió, qué otras opciones había y por qué las descarté. A esto se le conoce como registro de decisiones de arquitectura, o ADR por sus siglas en inglés.

Tabla 7. Registro de decisiones de arquitectura.

Clave	Decisión	Alternativa descartada y motivo	Estado
ADR-01	Construir el servidor con los servicios de Supabase: funciones, tiempo real y tareas programadas.	Un servidor propio con NestJS, Redis, Socket.IO y MinIO, como en el planteamiento inicial del módulo. Lo descarté porque suma cuatro herramientas que Bitlab-IT no tiene y que habría que mantener, cuando Supabase ya ofrece todo eso.	Aceptada
ADR-02	El agente pregunta cada pocos segundos si hay comandos pendientes.	Una conexión abierta todo el tiempo (WebSocket). Es más compleja; preguntar no abre puertos y funciona detrás de cualquier firewall, a cambio de unos segundos de espera.	Aceptada

Clave	Decisión	Alternativa descartada y motivo	Estado
ADR-03	Usar un catálogo cerrado de cerca de 50 comandos ligados a los tipos de alerta.	Una terminal para escribir cualquier comando. Por seguridad en el sector salud y por la hipótesis H4 me inclino por el catálogo, pero queda como decisión pendiente con la asesora.	Aceptada; por revisar con la asesora
ADR-04	Identificar a cada agente con su propia clave, guardada solo como huella.	Una sola llave para todos los agentes: si un equipo se viera comprometido, quedaría expuesta toda la base.	Aceptada
ADR-05	Construir el agente con lo que ya trae cada sistema.	PowerShell 7, Python, Node.js o Go: todos requieren instalar programas, lo que contradice el protocolo.	Aceptada
ADR-06	Ejecutar el agente con la cuenta SYSTEM en Windows y como LaunchDaemon en Mac.	Una cuenta con permisos mínimos, que no podría reiniciar servicios ni limpiar temporales; o sudo sin contraseña en Mac, que agrega riesgo.	Propuesta; por validar con la asesora
ADR-07	El módulo tiene tablas propias y solo lee las de la plataforma actual.	Modificar las tablas existentes, con el riesgo de afectar un sistema en operación.	Aceptada
ADR-08	Mostrar la consola con xterm.js dentro del proyecto de Lovable.	Un cuadro de texto sencillo, que no maneja colores, desplazamiento ni salidas largas. xterm.js es una librería del mismo proyecto de React, no una tecnología nueva.	Aceptada
ADR-09	Usar la API de Claude limitada al catálogo y sin ejecución automática.	Dejar que el modelo proponga cualquier comando (H4) o un agente de IA que actúe solo, que queda fuera del alcance.	Aceptada
ADR-10	Incluir modo de simulación en toda acción que modifique el equipo.	Confiar solo en la confirmación del técnico, que no deja ver el efecto antes de aplicarlo.	Aceptada

Fuente: Elaboración propia.

Riesgos del diseño

Tabla 8. Riesgos del diseño y cómo se reducen.

Riesgo	Cómo se reduce
Retraso en los accesos a la plataforma de Bitlab-IT.	Se trabaja en un proyecto propio de Supabase con la misma estructura y se adapta al recibir los accesos.
Que las tablas reales de equipos y alertas sean distintas a lo esperado.	Como el módulo tiene tablas propias (ADR-07), solo habrá que ajustar las relaciones.
Que la política de ejecución o el antivirus bloqueen al agente.	Scripts firmados y coordinación con el área de sistemas del cliente.
Que preguntar cada pocos segundos cargue la base con muchos equipos.	Tiempo de consulta configurable e índice para esa consulta; se medirá en el piloto.

Riesgo	Cómo se reduce
Que la inteligencia artificial sugiera un comando inseguro.	Catálogo cerrado, revisión de la respuesta y aprobación del técnico.

Fuente: Elaboración propia.

3. RESULTADOS DE LA ACTIVIDAD

Con esta actividad quedó integrado el entregable E1 completo. Además, durante la fase ya hubo avance práctico, que se resume en la Tabla 9.

Tabla 9. Avance del proyecto al cierre de esta actividad.

Parte	Avance
Organización	Proyecto BTL en Jira con épicas, tareas, entregables y sprints; repositorio en GitHub conectado a Jira.
Base de datos	Primera migración con 5 tablas, reglas de seguridad, funciones del agente, tiempo real y 13 comandos iniciales.
Agente para Windows	Versión de demostración en PowerShell que pregunta por comandos, los ejecuta, envía la salida en vivo y reintenta si falla.
Panel	Versión de demostración con inicio de sesión, selección de equipo, catálogo, modo de simulación y consola en vivo con xterm.js, y el prototipo completo «Bitlab Terminal».
Pendiente	Agente para Mac, tareas programadas e instaladores (E2 y E3), reglas por cliente (E4), función de ingesta (E5) y sugerencias de IA (E6).

Fuente: Elaboración propia.

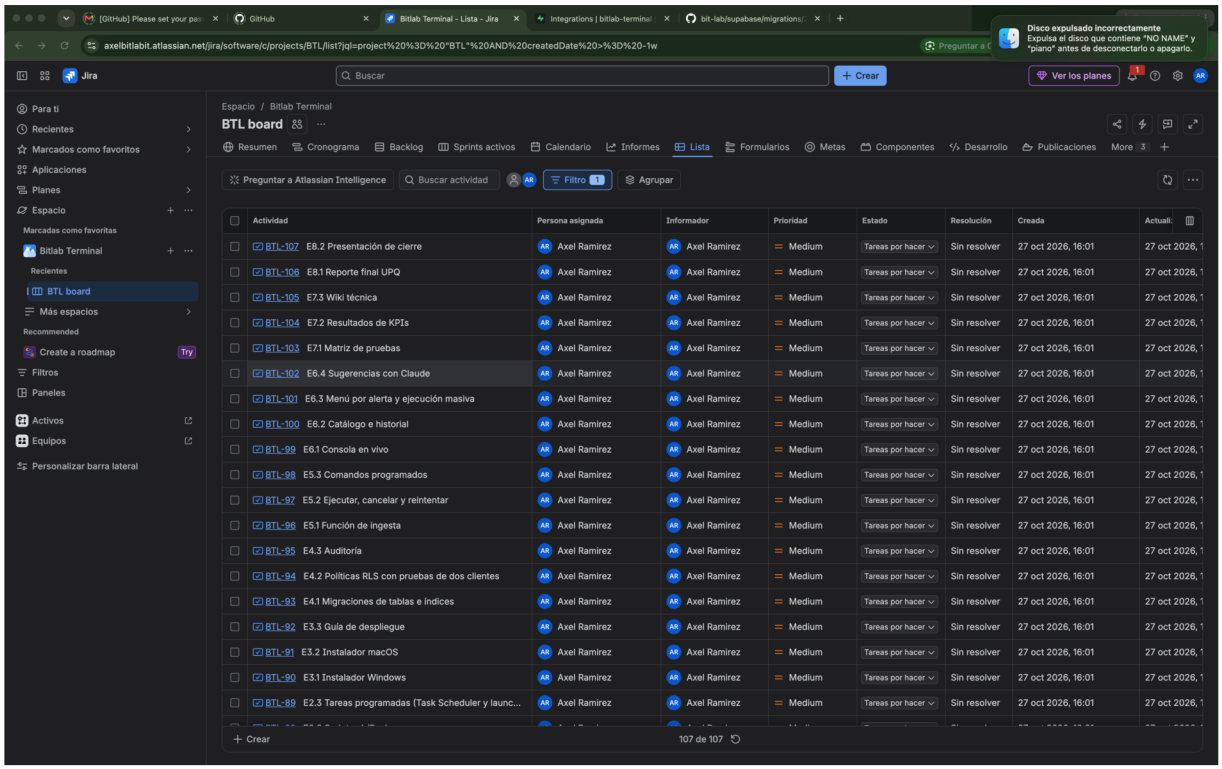


Figura 4. Tablero del proyecto BTL en Jira. *Fuente: Elaboración propia.*

https://github.com/axelbitlab/bit-lab

4. APROBACIÓN

Con su firma, la asesora externa aprueba este documento y las actividades A1.1 a A1.5 como base para las siguientes fases del proyecto, tomando en cuenta los pendientes de cada una.

Tabla 10. Firmas de aprobación.

Rol	Nombre	Firma	Fecha
Elaboró	Axel Gabriel Ramírez Ortiz Practicante, UPQ		
Aprobó	M.T.A. Sandra Lucía de la Fuente Bermúdez Asesora externa, Bitlab-IT		

Fuente: Elaboración propia.

REFERENCIAS

Anthropic. (2025). Tool use with Claude. Claude Docs. https://docs.claude.com/

Bitlab-IT. (2026). Protocolo de estadía profesional (versión 1.0).

Hevner, A. R., March, S. T., Park, J. y Ram, S. (2004). Design science in information systems research. MIS Quarterly, 28(1), 75–105. https://doi.org/10.2307/25148625

Supabase. (2024). Supabase documentation. https://supabase.com/docs